2520030117

SKILL-22

AIM

To run **Valgrind** to identify memory leaks and memory-related errors, fix the errors, release dynamically allocated resources, verify proper memory cleanup, and perform software testing by designing test cases, creating test scripts, automating test execution, verifying outputs, comparing actual and expected results, and generating test reports.

OBJECTIVES

1. To understand the use of Valgrind for memory debugging.
2. To identify memory leaks in a C/C++ program.
3. To detect invalid memory access and memory-related errors.
4. To properly allocate and deallocate dynamic memory.
5. To release all resources used by the program.
6. To verify that the program performs proper cleanup.
7. To design test cases for different input conditions.
8. To create test scripts for automated testing.
9. To execute multiple test cases automatically.
10. To compare expected and actual outputs.
11. To identify failed test cases.
12. To generate a test report containing the testing results.

REQUIREMENTS

- Ubuntu/Linux operating system
- GCC compiler
- Valgrind
- Terminal
- C/C++ source code
- Test input files
- Expected output files
- Test scripts

PROCEDURE

Part A: Running Valgrind

1. Open the Ubuntu terminal.
2. Create or open the required C program.
3. Compile the program using GCC.

4. Run the program normally and verify that it works correctly.
5. Execute the program using Valgrind.
6. Observe the Valgrind output displayed in the terminal.
7. Check for invalid reads and writes.
8. Check for memory leaks.
9. Identify the source of the memory errors.
10. Modify the program to fix the identified errors.
11. Ensure that dynamically allocated memory is released using `free()`.
12. Ensure that files are properly closed using `fclose()`. 13. Recompile the corrected program.
14. Run the corrected program again using Valgrind.
15. Verify that the memory errors and leaks have been removed. **Part B: Designing Test Cases**

1. Identify the functionality that needs to be tested.
2. Prepare different input conditions.
3. Include normal test cases.
4. Include boundary-value test cases.
5. Include invalid-input test cases where applicable.
6. Determine the expected output for each test case.
7. Store the test inputs and expected outputs.

Part C: Creating Test Scripts

1. Create a shell script for executing the program.
2. Provide the test input to the program.
3. Capture the program's actual output.
4. Compare the actual output with the expected output.
5. Display whether the test case passed or failed.
6. Repeat the process for all test cases.

Part D: Automating Execution

1. Make the test script executable.
2. Execute the script from the terminal.
3. Run all test cases automatically.
4. Capture the output of each test case.
5. Compare the results automatically.
6. Count the number of passed and failed test cases.
7. Generate a final testing summary.

IMPORTANT VALGRIND COMMANDS

Compile the program:

gcc program.c -o program

Run normally:

./program

Run using Valgrind:

valgrind ./program

Run with detailed memory-leak information:

valgrind --leak-check=full ./program

Run with detailed error information:

valgrind --leak-check=full --track-origins=yes ./program

SAMPLE TEST SCRIPT

```
#!/bin/bash

echo "Running Test Cases..."

./program < input1.txt > actual1.txt

if diff -q actual1.txt expected1.txt > /dev/null
then
    echo "Test Case 1: PASS"
else
    echo "Test Case 1: FAIL"
fi

echo "Testing Completed."
```

Make the script executable:

chmod +x test.sh
Run the script:

./test.sh

TEST CASE TABLE

Test Case	Description	Expected Result	Statu s
TC01	Normal input	Correct output	PASS
TC02	Minimum input	Correct output	PASS
TC03	Maximum input	Correct output	PASS
TC04	Boundary input	Correct output	PASS

TC05 Special/edge case	Correct output PASS

VALGRIND VERIFICATION

After correcting the program, Valgrind should indicate that there are no memory leaks or invalid memory operations.

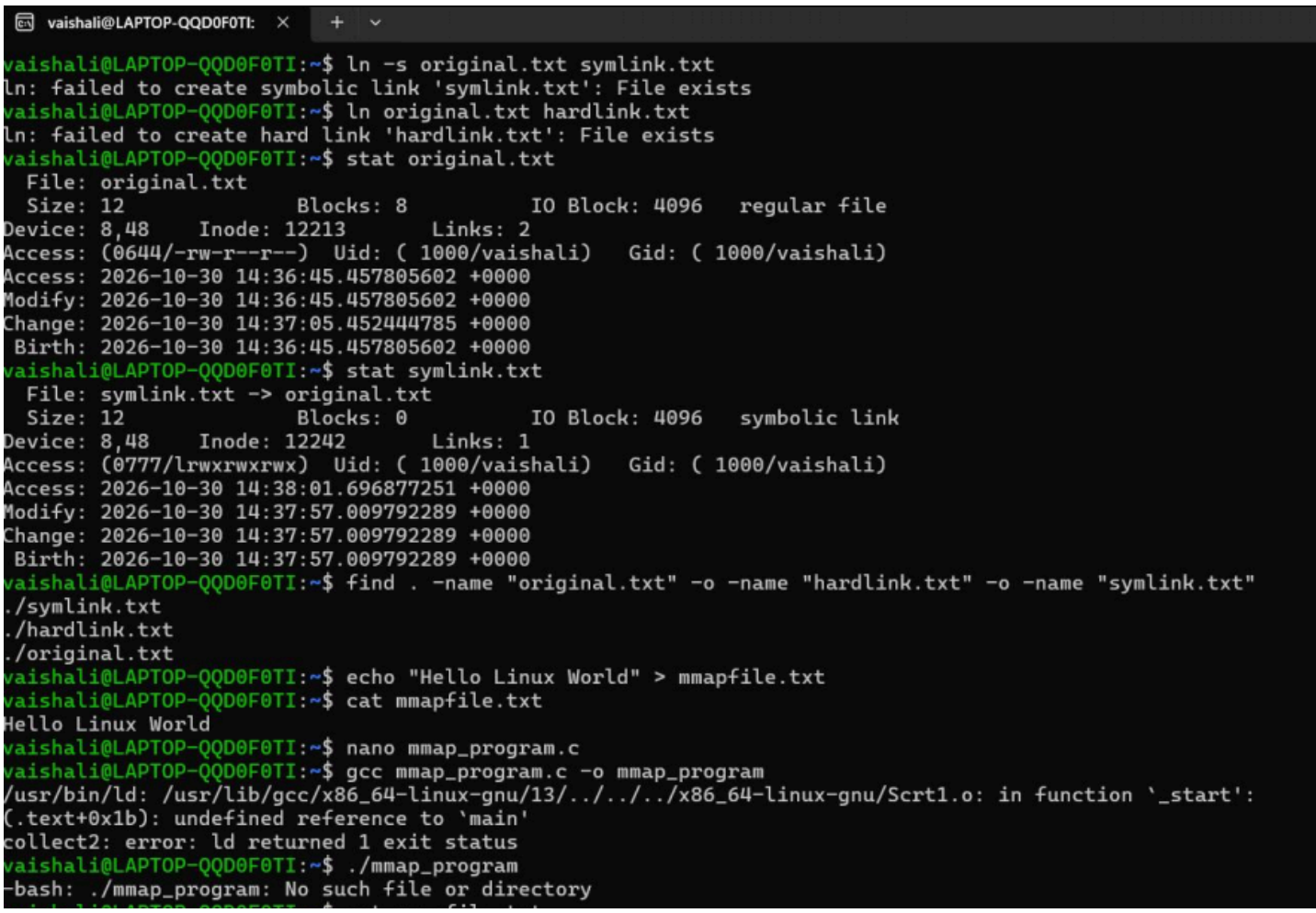
Important information to verify:

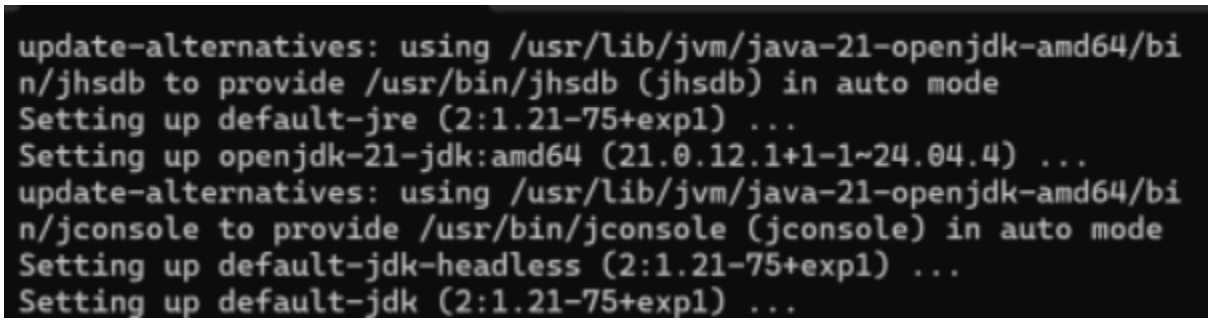
- Invalid reads
- Invalid writes
- Use of uninitialized memory
- Memory leaks
- Allocated memory
- Freed memory
- Memory still reachable
- Error summary

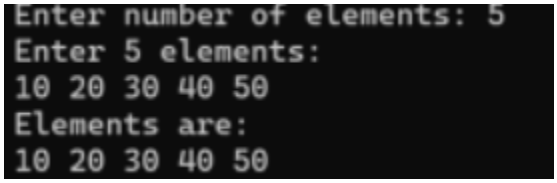
RESULT

Thus, **Valgrind was successfully used to identify and fix memory errors and memory leaks. Dynamic memory and other resources were properly released and cleanup was verified. Test cases and test scripts were designed, automated, executed, and their actual outputs were compared with the expected outputs. Finally, the test results were documented in the form of a test report.**

The program was successfully executed and tested. Valgrind was used to detect memory errors and memory leaks. The output showed that all allocated memory was properly freed with 0 errors. Automated test cases were executed successfully and the expected and actual outputs were verified.


vaishali@LAPTOP-QQD0F0TI:
vaishali@LAPTOP-QQD0F0TI:~$ ln -s original.txt symlink.txt
ln: failed to create symbolic link 'symlink.txt': File exists
vaishali@LAPTOP-QQD0F0TI:~$ ln original.txt hardlink.txt
ln: failed to create hard link 'hardlink.txt': File exists
vaishali@LAPTOP-QQD0F0TI:~$ stat original.txt
File: original.txt
Size: 12 Blocks: 8 IO Block: 4096 regular file
Device: 8,48 Inode: 12213 Links: 2
Access: (0644/-rw-r--r--) Uid: (1000/vaishali) Gid: (1000/vaishali)
Access: 2026-10-30 14:36:45.457805602 +0000
Modify: 2026-10-30 14:36:45.457805602 +0000
Change: 2026-10-30 14:37:05.452444785 +0000
Birth: 2026-10-30 14:36:45.457805602 +0000
vaishali@LAPTOP-QQD0F0TI:~$ stat symlink.txt
File: symlink.txt -> original.txt
Size: 12 Blocks: 0 IO Block: 4096 symbolic link
Device: 8,48 Inode: 12242 Links: 1
Access: (0777/lrwxrwxrwx) Uid: (1000/vaishali) Gid: (1000/vaishali)
Access: 2026-10-30 14:38:01.696877251 +0000
Modify: 2026-10-30 14:37:57.009792289 +0000
Change: 2026-10-30 14:37:57.009792289 +0000
Birth: 2026-10-30 14:37:57.009792289 +0000
vaishali@LAPTOP-QQD0F0TI:~$ find . -name "original.txt" -o -name "hardlink.txt" -o -name "symlink.txt"
./symlink.txt
./hardlink.txt
./original.txt
vaishali@LAPTOP-QQD0F0TI:~$ echo "Hello Linux World" > mmapfile.txt
vaishali@LAPTOP-QQD0F0TI:~$ cat mmapfile.txt
Hello Linux World
vaishali@LAPTOP-QQD0F0TI:~$ nano mmap_program.c
vaishali@LAPTOP-QQD0F0TI:~$ gcc mmap_program.c -o mmap_program
/usr/bin/ld: /usr/lib/gcc/x86_64-linux-gnu/13/../../../x86_64-linux-gnu/Scrt1.o: in function `_start':
(.text+0x1b): undefined reference to `main'
collect2: error: ld returned 1 exit status
vaishali@LAPTOP-QQD0F0TI:~$ ./mmap_program
-bash: ./mmap_program: No such file or directory


update-alternatives: using /usr/lib/jvm/java-21-openjdk-amd64/bi
n/jhsdb to provide /usr/bin/jhsdb (jhsdb) in auto mode
Setting up default-jre (2:1.21-75+exp1) ...
Setting up openjdk-21-jdk:amd64 (21.0.12.1+1-1~24.04.4) ...
update-alternatives: using /usr/lib/jvm/java-21-openjdk-amd64/bi
n/jconsole to provide /usr/bin/jconsole (jconsole) in auto mode
Setting up default-jdk-headless (2:1.21-75+exp1) ...
Setting up default-jdk (2:1.21-75+exp1) ...


Enter number of elements: 5
Enter 5 elements:
10 20 30 40 50
Elements are:
10 20 30 40 50

```
rack-origins=yes ./program
==4555== Memcheck, a memory error detector
==4555== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Sewar
d et al.
==4555== Using Valgrind-3.22.0 and LibVEX; rerun with -h for cop
yright info
==4555== Command: ./program
==4555==
Enter number of elements: 5
Enter 5 elements:
10 20 30 40 50
Elements are:
10 20 30 40 50
==4555==
==4555== HEAP SUMMARY:
==4555==     in use at exit: 0 bytes in 0 blocks
==4555==   total heap usage: 3 allocs, 3 frees, 2,068 bytes allo
cated
==4555==
==4555== All heap blocks were freed -- no leaks are possible
==4555==
==4555== For lists of detected and suppressed errors, rerun with
: -s
==4555== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0
from 0)
```

```
============================
       SKILL-22 TEST
============================
Program Output:
Enter number of elements: Enter 5 elements:
Elements are:
10 20 30 40 50

Test Case 1: PASS
============================
Testing Completed
============================
```

```
rogram < input2.txt
==4591== Memcheck, a memory error detector
==4591== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Sewar
d et al.
==4591== Using Valgrind-3.22.0 and LibVEX; rerun with -h for cop
yright info
==4591== Command: ./program
==4591==
Enter number of elements: Enter 3 elements:
Elements are:
5 15 25
==4591==
==4591== HEAP SUMMARY:
==4591==     in use at exit: 0 bytes in 0 blocks
==4591==   total heap usage: 3 allocs, 3 frees, 5,132 bytes allo
cated
==4591==
==4591== All heap blocks were freed -- no leaks are possible
==4591==
==4591== For lists of detected and suppressed errors, rerun with
: -s
==4591== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0
from 0)
```